

Online SNN Training for Temporal Coding

YUHAO LEONG, National University of Singapore, Singapore

Spiking neural networks are an emerging deep learning architecture closely modelled after biological brain activity, where signals are propagated through spikes in continuous time. This enables computationally efficient spatio-temporal reasoning over classical artificial neural networks on analog neuromorphic hardware. As standard backpropagation cannot be applied for SNNs, a wide variety of training methods have been proposed. Most of these methods, however, perform parameter updates through discretized time steps, which we hypothesize to be detrimental for learning precise temporal features.

1 Introduction

Having achieved remarkable successes across various fields in the post-transformer era, Deep Learning (DL) is currently experiencing a time of unprecedented usage and commercialization. To meet the ever-growing demands of consumers, firms offering related services have constantly been allocating large sums of money into scaling up technological capabilities, through a combination of better hardware and more power-intensive computing facilities, leading to concerns about the overwhelming energy consumption of these facilities or the scalability of hardware such as GPUs. There has also been increased skepticism on the phenomenon known as Moore’s Law — a historical trend where the number of processors on an IC chip doubles approximately every two years, with some predicting that a boundary of digital hardware scalability may soon be reached.

Artificial neural networks (ANNs) form the foundation of the vast majority of applications in DL today, attributing its current success to substantial prior research and development. Universal approximation theorems (UATs) for ANNs provide a theoretical guarantee that any function can be approximated with some ANN, serving as a backbone for most modern research. Characterized by layers of ‘neurons’, whose activations are given by some continuous function, ANNs operate on forward passes at fixed time steps, where signals are transmitted in bulk from one layer to the next.

Spiking neural networks (SNNs) is an alternative architecture for DL that use discrete spikes in continuous time as activations, in a way that further matches biological synaptic activity. While still relatively new, SNNs feature some advantages over ANNs: **(1)** sparse instead of dense activations require less computational power if used with specialized neuromorphic hardware; **(2)** greater compatibility with analog circuits means less computational burden of digital abstractions; **(3)** continuous spiking time is also encoded as temporal information, extending the range of applications to modelling time-dependent systems. However, formal UATs for SNNs have been unheard of until recently in 2022 [13] and 2025 [2], which may drive interest towards this new architecture. In the field of SNNs, a significant area of research is developing training algorithms that not only converge well but also bear the following qualities: **(I)** handle temporal information properly; **(II)** maintain sparsity and low

space complexity; **(III)** exhibit local and online learning. These qualities ensure that training algorithms can be directly and effectively implemented on physical systems while still maintaining a clear advantage over ANNs.

While many training algorithms have been developed, most do not abide by the above qualities, indicating a long road ahead towards analog neuromorphic implementation. We hereby propose a research framework to develop an algorithm possessing all three qualities, which could pave the way for a shift from the digital to analog age of AI, and also provide insight into the true learning process of biological neurons.

2 Background

SNNs often share the same layer-wise topology as ANNs (e.g., feed-forward, convolutional), but differ in terms of activation function: Typical ANNs have neuron activations given by $h = \sigma(\mathbf{w}\mathbf{x} + b)$, while SNNs replace this with a system modelled by a time-based dynamical system, most commonly being the Leaky Integrate-and-Fire (LIF) model. We provide the simplified mechanism of a single LIF neuron, where physical terms are deliberately abstracted away:

Discretized LIF

- 1. Integrate + Leak** At each time step, the potential $\tilde{V}_t$ is updated based on decay α and the weighted sum of input activations.

$$\tilde{V}_t = \alpha V_{t-1} + \mathbf{w}\mathbf{x}_t$$

- 2. Spike Function** A step function then determines if a spike is fired when a threshold V_{th} is reached.

$$S_t = \begin{cases} 1 & \text{if } \tilde{V}_t \geq V_{th} \\ 0 & \text{otherwise} \end{cases}$$

- 3. Reset** If a spike is fired, the post-activation potential V is then reset to zero.

$$V_t = (1 - S_t) \tilde{V}_t$$

Continuous-Time LIF

- 1. Integrate + Leak** The potential $V(t)$ is modeled by a first-order ODE with decay α and the weighted sum of input signals.

$$\frac{dV(t)}{dt} = (1 - \alpha)V(t) + \mathbf{w}\mathbf{x}(t)$$

- 2. Spike Function** A spike signal $S(t)$ is generated at the time t_f when a threshold V_{th} is reached.

$$S(t) = \delta(t - t_f) \quad \text{if } V(t_f) = V_{th},$$

where $\delta(t)$ is the Dirac delta function.

- 3. Reset** After firing, the potential is immediately reset to zero.

$$V(t_f^+) = 0$$

In addition to weights $\mathbf{W}$, the decay factor α and the firing threshold V_{th} may also serve as parameters to be trained. Temporal dynamics and the non-differentiable spike function make backpropagation particularly tricky to implement. [10] has demonstrated that ignoring the temporal dynamics for backpropagation in SNNs leads to reduced performance.

2.1 Encoding Input Data

Just as spikes form the input activations for each neuron, input information must also be encoded as spike trains, for which there are two main categories of encoding schemes [4]. The scheme used strongly influences the functionality of the model, since the model learns to generate internal signals bearing similar characteristics of input signals.

2.1.1 Rate Coding. Data is encoded such that the intensity of input stimuli directly corresponds to the spike rate of the resultant spike train. While straightforward, SNNs that are trained on rate-coded inputs have difficulty picking up temporal patterns that rely on highly precise spike times, and do not provide much advantage over ANNs due to the spike rates in SNNs becoming functionally similar input weights in ANNs.

2.1.2 Temporal Coding. Data is encoded in sparse, specific patterns, with a notable example being Time-to-First-Spike. These schemes offer an energy advantage over rate coding due to sparse activations, while incorporating the timing of spikes as significant information carriers. However, training is generally more difficult as precise continuous-valued temporal information would have to be captured.

2.2 Training Methods

2.2.1 Backpropagation Through Time (BPTT). A popular approach to training SNNs uses a combination of BPTT and surrogate gradients. When simplified to only consider discrete time steps, SNNs have been shown in [7] to be equivalent to recurrent neural networks, for which BPTT is a standard training algorithm. The issue of non-differentiable spikes is also mostly overcome by using surrogate or "pseudo" gradients as a replacement for BPTT to take place. While this method has been the gold standard for many tasks, it has several weaknesses: (1) unfolding in time causes its space complexity to scale with both the number of neurons and time steps; (2) the discretization of time may push SNNs to become functionally rate-based, since spike timings are not accounted for during backpropagation and therefore temporal information is lost.

2.2.2 Event-Driven. Methods in this category only perform parameter updates based on event (spike) times, instead of discrete time steps, thus capturing temporal information. Naturally, these methods would be more suitable for SNNs implemented on analog (continuous-valued) hardware, with emphasis on temporal coding. However, when simulated on discrete time, their advantage over other methods may not be apparent or even lost. Methods include [6, 9].

2.2.3 Online Learning. Typically, online learning refers to methods that learn "on the fly", without first having to wait for future signals. In the context of SNNs which operate in continuous time, this

refers to methods that readily update parameters during the forward pass without having to wait for backpropagated error signals, thus incurring no latency in learning. Methods include [1, 3, 11].

2.2.4 Local Learning. Strictly local learning methods perform parameter updates to a neuron based on pre- and post-synaptic signals, without the need for other information. Semi-local methods relax this rule by including global variables, such as a reward factor that does not involve heavy computation. Local learning also avoids the weight transport problem of backpropagation, which is thought to be biologically unfeasible. Methods include [1, 5].

2.2.5 Real-Time Recurrent Learning (RTRL). RTRL is an online alternative to BPTT, performing forward-mode autodifferentiation that incurs $O(N^4)$ time complexity with respect to the number of parameters. Due to this apparent flaw, it serves little practical purpose, but sets a precedent for algorithms with much faster time complexity, such as e-prop [1] and OSTL [3]. These methods use a local eligibility trace for each neuron that stores the importance of past time states, combined with a global learning signal to perform approximate weight updates matching RTRL/BPTT.

2.2.6 Spike-Timing-Dependent Plasticity (STDP) and Hebbian Learning. 2-factor Hebbian learning rules are processes where spatio-temporal relationships can be learned solely by observing the pre- and post-synaptic activities of a neuron, then performing weight adjustments based on this observation. STDP is one such rule, where if a neuron receives an incoming signal shortly before it fires, the corresponding weight of the input signal is strengthened. On its own, 2-factor Hebbian Learning rules are unsupervised and insufficient to train networks to perform tasks. Existing training methods use a variant called 3-factor Hebbian learning (with reward/feedback signals) or impose other conditions allowing supervised learning [1, 5, 8, 11].

3 Research Problem

A vast majority of training algorithms for SNNs simplify training by only performing parameter updates in discrete time steps [1, 3, 7, 10–12], which may forego precise temporal information and encourage the model to become rate-based, or in other words, focus on spike frequency rather than spike patterns in input data. A trade-off involves taking smaller time steps in exchange for having to store more states across time. These algorithms are also not faithful to the sparse, event-driven nature of SNN inference, as dense full backpropagation has to be carried out during the training stage. Instead, we look towards methods such as EventProp [9] as a baseline.

Instead of unfolding the network in time to perform a variant of BPTT, EventProp is the first method, to our knowledge, that treats the individual neurons as what they are: hybrid dynamical systems featuring both discrete and continuous components. It uses a technique from optimal control theory known as the adjoint state method, which applies to ordinary differential equations (ODEs). Notably, it computes the change of the ODE solution with respect to an independent variable, similar to finding the partial derivative of a function in backpropagation. The adjoint method achieves this by solving a dual problem known as the adjoint ODE, which can

be readily implemented on analog circuits. EventProp avoids the problem of non-differentiable spikes entirely by instead treating each spike as events in continuous time, and applying the adjoint method across spikes. Discontinuities such as $V(t_f^+) = 0$ are also rigorously handled by imposing jump conditions. Therefore, we are certain that EventProp can compute the exact gradients of SNNs accounting for temporal dynamics.

However, EventProp lacks the quality of being local, and more importantly, online. To calculate gradients, the forward functionality of the network would first have to be paused at some time $t = T$, as only then can the system from $t = 0$ to $t = T$ be analyzed. The states of each spike, such as time and incoming synaptic current, would also have to be stored, incurring an $O(S)$ memory overhead, where S is the number of spikes. On analog hardware implementations, this would require dynamic allocation of memory that is not ideal, as it might break the continuous-time dynamics of SNN inference.

4 Methodology

We propose a possible framework for training SNNs that implements the adjoint state method while being fully online, semi-local, and with space complexity agnostic to the number of events, with a goal to achieve temporal coding performance close to EventProp. Throughout the duration of our research, we will take on a continuous-time standpoint to approach problems. One example is overcoming temporal credit assignment, which will enable the model to relate past events to current and future states, by formulating eligibility traces that may enable online learning. This is necessary as simplifying training dynamics to discrete windows of time was hypothesized to be a pitfall of most algorithms. Furthermore, we will also seek to investigate the connection of the proposed method to Hebbian learning, as this would also serve as an indicator of locality and energy efficiency for neuromorphic hardware implementation.

Unfortunately, resources for training SNNs are scarce. Modern processing units manufactured for DL focus almost exclusively on dense and digital computation, favoring ANNs while depriving SNNs of their inherent advantage of being sparse and energy-efficient. While a handful of companies offer specialized neuromorphic chips (e.g., Intel Loihi, IBM TrueNorth), they are often reserved for internal research purposes and are not commercially available. Furthermore, neuromorphic chips differ greatly from one another in terms of architecture, which may cause inaccurate comparisons as one chip might favour certain training algorithms over another. Most neuromorphic hardware are also completely digital, going against the analog nature of our research topic. Apart from specialized hardware, a popular alternative directly simulates SNNs on standard processors using DL libraries like Pytorch. In either case, the true theoretical performance and energy efficiency would not be accurately reflected, so implementation may very well serve as a proof-of-concept. For this project, we also explore the possibility of being granted access to neuromorphic hardware.

5 Timeline

We estimate the duration of the research project to span approximately 30 weeks, split across 3 phases.

5.1 Phase 1: Weeks 1 - 9

The first phase of the research project involves gathering and replicating prior methods into a single pipeline for initial testing. As SNNs are not standard for most devices, it is important to first configure a working pipeline for training SNNs with the standard BPTT + surrogate gradient approach. Once complete, we can move on to replicating or integrating other methods into the training pipeline, such as, but not limited to, EventProp [9] and OSTL [3]. Then, we run some initial simulations to hopefully gain some insights into the different methods used, such as determining optimal hyperparameters and measuring computational efficiency.

Throughout the first phase, workload is deliberately kept low, allowing buffer time to review mathematical concepts such as numerical ODE methods, dynamical systems, and optimization. At the same time, we will also examine literature in the scope of RTRL and online learning, and neural ODEs, which are related to SNNs by the applicability of the adjoint method. By the end of the first phase, we will identify multiple candidate formulations for our online algorithm.

Tasks	DECEMBER					JANUARY			
	1	2	3	4	5	6	7	8	9
Environment setup									
Review methods									
Replicate methods									
Local simulation									
Research report + code									

Fig. 1. Preliminaries

5.2 Phase 2: Weeks 10 - 19

The second phase starts with an initial formulation for our online, adjoint-based training algorithm. The first six weeks will be dedicated to implementing the algorithm, occasionally performing small-scale tests to verify convergence. Then, we move on to collecting empirical data by running larger simulations across different hyperparameter settings, model complexities, and input encoding schemes, before documenting our findings. The corresponding research report will also carry greater significance, since it will simultaneously be used as an application to access advanced neuromorphic hardware, which may be granted for research purposes.

5.3 Phase 3: Weeks 20 - 30

By the start of the last phase, we should already have a functional algorithm that may only require some optimization or refining. In the event that access to specialized neuromorphic hardware is granted, it is likely that our code would still have to be adapted to a different framework, for which a 3-week buffer period is given.

Tasks	FEBURARY				MARCH				APRIL	
	10	11	12	13	14	15	16	17	18	19
Mathematical Formulation										
Implementation										
Local simulation										
Research report + code										
Neuromorphic chip application										

Fig. 2. Algorithm Formulation and Testing

Otherwise, large-scale simulation can be shifted forward. Then, we compare our algorithm with a variety of different algorithms for performance in both rate coded and temporal coded inputs. Finally, we establish formal mathematical derivations to prove the convergence of our algorithm.

Tasks	APRIL			MAY				JUNE		
	20	21	22	23	24	25	26	27	28	30
Algorithm refinement										
(Neuromorphic?) Simulation										
Data collection + processing										
Proof development										
Final manuscript										

Fig. 3. Data Collection and Finalization

References

- [1] Guillaume Bellec, Franz Scherr, Anand Subramoney, Elias Hajek, Darjan Salaj, Robert Legenstein, and Wolfgang Maass. A solution to the learning dilemma for recurrent networks of spiking neurons. *Nat. Commun.*, 11(1):3625, July 2020.
- [2] Umberto Biccari. Spiking neural networks: a theoretical framework for universal approximation and training, 2025. URL <https://arxiv.org/abs/2509.21920>.
- [3] Thomas Bohnstingl, Stanislaw Woźniak, Wolfgang Maass, Angeliki Pantazi, and Evangelos Eleftheriou. Online spatio-temporal learning in deep neural networks, 2020. URL <https://arxiv.org/abs/2007.12723>.
- [4] Kai Malcolm and Josue Casco-Rodriguez. A comprehensive review of spiking neural networks: Interpretation, optimization, efficiency, and best practices, 2023. URL <https://arxiv.org/abs/2303.10780>.
- [5] Erwann Martin, Maxence Ernoult, Jérémie Laydevant, Shuai Li, Damien Querlioz, Teodora Petrisor, and Julie Grollier. Eqspike: Spike-driven equilibrium propagation for neuromorphic implementations, 2021. URL <https://arxiv.org/abs/2010.07859>.
- [6] Emre O. Neftci, Charles Augustine, Somnath Paul, and Georgios Detorakis. Event-driven random back-propagation: Enabling neuromorphic deep learning machines. *Frontiers in Neuroscience*, 11, June 2017. ISSN 1662-453X. doi: 10.3389/fnins.2017.00324. URL <http://dx.doi.org/10.3389/fnins.2017.00324>.
- [7] Emre O. Neftci, Hesham Mostafa, and Friedemann Zenke. Surrogate gradient learning in spiking neural networks, 2019. URL <https://arxiv.org/abs/1901.09948>.
- [8] Peter O'Connor, Efstratios Gavves, and Max Welling. Training a spiking neural network with equilibrium propagation. In Kamalika Chaudhuri and Masashi Sugiyama, editors, *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 1516–1523. PMLR, 16–18 Apr 2019.
- [9] Christian Pehle, Luca Blessing, Elias Arnold, Eric Müller, and Johannes Schemmel. Event-based backpropagation for analog neuromorphic hardware, 2023. URL <https://arxiv.org/abs/2302.07141>.
- [10] Yujie Wu, Lei Deng, Guoqi Li, Jun Zhu, and Luping Shi. Spatio-temporal back-propagation for training high-performance spiking neural networks. *Frontiers in Neuroscience*, 12, May 2018. ISSN 1662-453X. doi: 10.3389/fnins.2018.00331. URL <http://dx.doi.org/10.3389/fnins.2018.00331>.
- [11] Mingqing Xiao, Qingyan Meng, Zongpeng Zhang, Di He, and Zhouchen Lin. Online training through time for spiking neural networks, 2022. URL <https://arxiv.org/abs/2210.04195>.
- [12] Friedemann Zenke and Surya Ganguli. Superspike: Supervised learning in multi-layer spiking neural networks. *Neural Computation*, 30(6):1514–1541, June 2018. ISSN 1530-888X. doi: 10.1162/neco_a_01086. URL http://dx.doi.org/10.1162/neco_a_01086.
- [13] Shao-Qun Zhang and Zhi-Hua Zhou. Theoretically provable spiking neural networks. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 19345–19356. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/7abbc05a5d55157ede410bb718e32d7-Paper-Conference.pdf.